

AudiTrim: A Real-time, General, Efficient, and Low-overhead Data Compaction System for Intrusion Detection

Hongbin Sun
Zhongguancun Laboratory
Beijing, China
sunhb@mail.zgclab.edu.cn

Su Wang*
Zhongguancun Laboratory
Beijing, China
wangsu@zgclab.edu.cn

Zhiliang Wang*
Tsinghua University
Beijing, China
wzl@cernet.edu.cn

Zheyu Jiang
Tsinghua University
Beijing, China
jiangzy23@mails.tsinghua.edu.cn

Dongqi Han
Beijing University of Posts and
Telecommunications
Beijing, China
handongqi@bupt.edu.cn

Jiahai Yang
Tsinghua University
Beijing, China
yang@cernet.edu.cn

ABSTRACT

Recently enterprises and governments face escalating APT attacks, leading to significant economic losses. APT attacks often persist for extended periods, necessitating the storage of extensive audit logs for effective detection. To reduce data storage overhead, enterprises commonly adopt compression strategies. However, efficient compression strategies may introduce additional query overhead. Existing approaches propose data reduction algorithms, but these methods can compromise data integrity, rendering current attack investigation and anomaly-based intrusion detection ineffective.

To address these difficulties, we present AUDITRIM, a system that ensures real-time, general, efficient, and low-overhead data compaction without compromising attack investigation and anomaly-based intrusion detection. It efficiently reduces log sizes without impacting user experiences, achieving real-time compaction and adaptable deployment on different operating systems. AUDITRIM employs two strategies: 1) Data Reduction: By analyzing the types of duplicate edges, our data reduction approach not only considers a broader range of scenarios for redundant edges compared to previous methods but also enhances the efficiency of data reduction. 2) Data Compression: By aggregating log information at the server-side and training a compression model, we facilitate a data compression algorithm that ensures ease of querying. Both strategies meet real-time, low-overhead, and general requirements, fulfilling enterprise data storage needs. The final compaction ratio reaches $26\times$ – $65\times$.

CCS CONCEPTS

• Security and privacy → Intrusion detection systems.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

RAID 2024, September 30–October 02, 2024, Padua, Italy

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0959-3/24/09

<https://doi.org/10.1145/3678890.3679048>

KEYWORDS

data compaction, attack investigation, intrusion detection

ACM Reference Format:

Hongbin Sun, Su Wang, Zhiliang Wang, Zheyu Jiang, Dongqi Han, and Jiahai Yang. 2024. AudiTrim: A Real-time, General, Efficient, and Low-overhead Data Compaction System for Intrusion Detection. In *The 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024)*, September 30–October 02, 2024, Padua, Italy. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3678890.3679048>

1 INTRODUCTION

Today, network attacks have become increasingly frequent. More and more Advanced Persistent Threat (APT) groups have emerged. APT attacks differ from traditional forms of network attacks as they are characterized by their stealthiness, advanced techniques, persistence, targeted approach[36]. Attackers often employ social engineering techniques and zero-day vulnerabilities to infiltrate target hosts. Therefore, compared to other network attacks, APT attacks are difficult to detect and defend against. Recently, host-based intrusion detection system (HIDS) based on provenance graph has been considered effective approach to prevent network attacks, especially APT attacks.

As shown in Figure 1, the HIDS process involves capturing audit logs on hosts, uploading the logs to server, performing intrusion detection on server, and storing the logs and alarms for attack investigation. Currently, there are well-established tools for data collection. Linux Audit[3] and Windows Event Tracing for Windows (ETW)[2] are both operating system-level log frameworks that capture kernel-level events and provide high reliability. Linux Audit is a powerful auditing framework for tracking and recording various activities in Linux systems. It can monitor events such as file and directory access, system call, process creation and termination, user logins and logouts, and more. Windows ETW is used to record various events in Windows operating systems, providing efficient data collection and processing mechanisms.

An active host can generate up to 5GB of logs per-day[23]. To detect long-duration APT attacks, storing logs for extended periods is necessary, but it can result in significant storage overhead for servers. Therefore, enterprises often need to consider the trade-off of log retention duration and typically retain logs for 3 months to 6 months. However, for detecting and tracing APT attacks, the

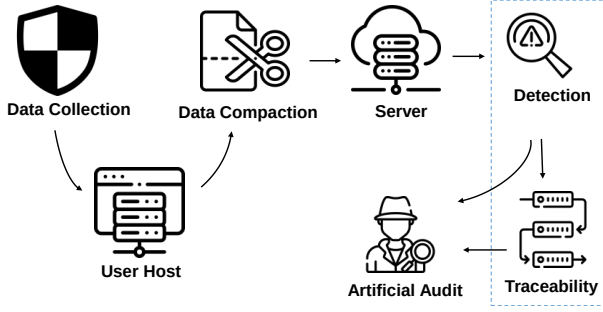


Figure 1: Host-based intrusion detection process.

retained log duration is insufficient. Moreover, since the detection process occurs on the server side, users need to upload logs to the server, which consumes a significant amount of bandwidth on user hosts. Additionally, considering the large number of employees in an enterprise, this also places a significant burden on server side bandwidth. Therefore, data compaction work can effectively improve the efficiency of intrusion detection, reduce storage overhead on servers, and minimize bandwidth consumption on both hosts and servers.

Based on the HIDS process, we propose a data reduction algorithm employed at the host side. The data reduction algorithm faces the following challenges in practical scenarios: efficiency, generality, low consumption, and real-time performance.

- **Efficiency.** Many existing methods lack the ability to achieve high compaction ratios or are limited to specific scenarios, failing to meet practical requirements[14, 25, 29, 35]. Given the large scale of audit logs, an efficient data reduction method should be developed to effectively reduce data storage costs.
- **Generality.** Since the pruned logs need to undergo intrusion detection in the server, the reduction process should not compromise the integrity of the logs or reduce detection efficiency. It should be compatible with existing detection algorithms.
- **Low consumption.** As the data reduction process occurs on user hosts, complex data compaction algorithms or methods that involve detection for data reduction can consume significant host resources, negatively impacting user experience and productivity. Therefore, when designing the data reduction algorithm, it is important to avoid excessive memory resource usage, with a typical range of 2% to 3% memory occupancy having minimal impact on user experience.
- **Real-time performance.** Real-time performance is essential for timely attack detection and mitigation. Intrusion detection needs to be performed in real-time, and after data collection, the logs should be uploaded from the host side in a timely manner. Therefore, reduction methods that require the entire logs as input are not suitable, and real-time reduction methods are necessary.

To address these challenges, we propose AUDITRIM, a universal, efficient, low-cost, and real-time data compaction system. This model consists of two parts: data reduction and data compression. In the first part, direct causality dependency is defined, and five scenarios of duplicate edges occurring are summarized. Redundant edges with duplicate semantics are removed to ensure efficient

reduction. The inclusion of a time window strategy enhances the generality, real-time performance, and low consumption of data reduction. In the second part, compression is achieved through the statistical analysis of common properties and attributes in the audit logs and Huffman[21] coding. Combined with the data reduction strategy, a compaction ratio of 60× is achieved. The main contributions of this paper are as follows:

- We identify five scenarios of redundant edges by analyzing direct causality dependency and thus implement an efficient data reduction strategy. Compared to existing work, the data reduction efficiency is improved by 2×.
- Compared to existing works, this paper is the first to consider the impact of data reduction on intrusion detection algorithms. Through process behavior analysis, we introduce the time window strategy to ensure the efficiency of intrusion detection methods. Furthermore, by replicating existing work and conducting experimental analysis, we demonstrate that the reduction algorithm can be directly integrated with existing intrusion detection algorithms.
- By combining reduction method with compression method, we implement a general, efficient, real-time, and low-overhead data compaction system, achieving compaction ratios of 26×-65×. AUDITRIM is available online at <https://github.com/AudiTrim/AudiTrim/tree/main>.

2 BACKGROUND

2.1 Provenance Graph

Audit log consists of source vertex, destination vertex, event type and timestamp. The connection between source vertex and destination vertex forms a dependency graph. When a process writes to a file and then reads from it, a cycle is created in the dependency graph. CamFlow[32] proposes updating vertex when it transitions from an information inflow state to an information outflow state, thereby generating a directed acyclic graph (DAG) referred to as data provenance graph. In provenance graph, vertices represent entities such as processes and files, while edges represent system behaviors, and the edge direction indicates the direction of information flow. Research studies [4, 5, 13] have shown that compared to traditional network traffic and initial system audit logs, data provenance graphs contain rich contextual information, which can be used to detect long-term system behavior and aid in attack reconstruction. The provenance graph not only avoids the generation of cycles but also captures time information, thus avoiding false dependency. As shown in Figure 2, (a) represents the dependency graph, and (b) represents the data provenance graph. Disregarding time information, in the dependency graph, process P_3 would pass the information to process P_2 through file F. However, since w_4 occurs later than r_2 , there is no information transmission from P_3 to P_2 . In the provenance graph, false dependencies are eliminated. Therefore, current related works use the provenance graph as input.

However, when the dependency graph contains a large number of duplicate events, it can lead to multiple updates of vertices in the provenance graph. We provide an example in Figure 3, when employees modify files, numerous read and write events are generated. In order to avoid false dependency, provenance graph performs

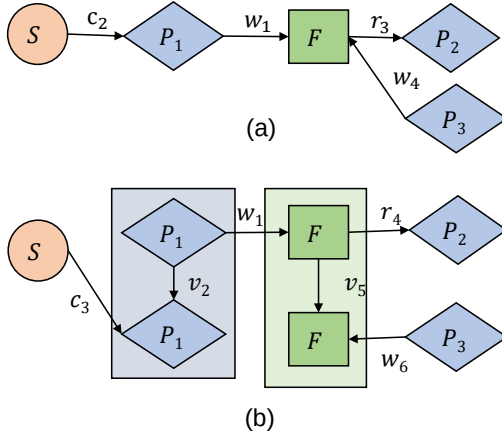


Figure 2: (a) is dependency graph, (b) is provenance graph of (a). S, P, F represent socket, process and file. w, r, c, v stand for event type of write, read, connect and version. The subscripts i represent event ordering.

vertex updates after each write event. These updates are unnecessary and can complicate the graph's structure. w_1 , originally 1-hop event of F , has now transformed into a 3-hop event of F'' in the data provenance graph. This can have a negative impact on anomaly models that rely on graph algorithms. As shown in Figure 3(c), if data reduction is applied to the logs before generating the provenance graph, it can reduce the frequency of node updates and enhance the efficiency of graph-based anomaly detection algorithms.

2.2 Causality Dependency

If there is information transmission between two events, they are referred to as causality dependency. If two events have causality dependency, with one event's source vertex being the destination of the other event, they have direct causality dependency. We define an event as $e = (u, v, t)$, where u and v are vertices. u is the source vertex, v is the destination vertex and t is the timestamp of event occurrence. For two events $e_1(u_1, v_1, t_1)$ and $e_2(u_2, v_2, t_2)$ have direct causality dependency, the following conditions are satisfied:

$$v_1 = u_2, t_1 < t_2 \text{ or } u_1 = v_2, t_2 < t_1.$$

For two events $e_1(u_1, v_1, t_1)$ and $e_2(u_2, v_2, t_2)$ that have causality dependency, if e_2 has causality dependency with another event e_3 , then if $t_1 < t_2 < t_3$ or $t_3 < t_2 < t_1$, e_1 and e_3 also have causality dependency. Taking $t_1 < t_2 < t_3$ as an example, since $t_1 < t_2$, it means that e_1 event occurs first. Due to the causality dependency between e_1 and e_2 , there is information flow from e_1 to e_2 . Similarly, the information from e_2 can be transmitted to e_3 . Thus, in the process of information flow, the information from e_1 is transmitted to e_3 through e_2 , indicating the causality dependency between e_1 and e_3 . As shown in Figure 2(a), w_1 and r_3 have causality dependency, w_4 and r_3 do not have causality dependency.

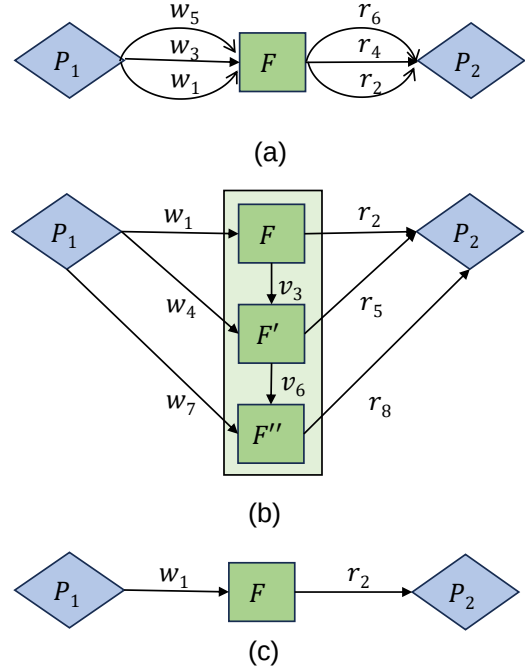


Figure 3: (a) is dependency graph with duplicate edges, (b) is provenance graph of (a) with multiple version events. (c) is provenance graph after data reduction.

3 RELATED WORK

The main process of intrusion detection consists of three parts: data collection, data storage, and intrusion detection. Since this work proposes a general data reduction strategy that does not decrease the efficiency of existing intrusion detection methods, in order to better demonstrate this point, this section will not only introduce the related work on data storage but also discuss the work on detection part.

3.1 Data Reduction

In Section 1, the challenges faced by data reduction methods are introduced. In this section, existing work will be categorized based on their issues.

3.1.1 Efficiency. NodeMerge [35] and LogApprox[29] analyze log generation and identify a pattern of events related to read-only system file interactions that occur during program initialization. Nodemerge achieves node merging by analyzing and consolidating these system file nodes. LogApprox accomplishes the consolidation of these events by generating minimized regular expressions to match these files. Due to their limited application scenarios, their compression efficiency is low[23]. [20, 41, 46] have each proposed the Causality Preserving Reduction (CPR), the Full Dependence Preservation (F-DPR) reduction, and the Maintaining Global Semantics (GS) reduction based on forensic analysis. They achieve merging equivalent repetitive events by analyzing whether the results of trace and propagation on entities in the provenance graph are consistent. However, due to the lack of a systematic analysis of

causality dependency, they have only achieved partial reduction of redundant edges. To improve pruning efficiency, they propose the Process-centric Causality Approximation Reduction (PCAR)[41], Source Dependence Preservation (S-DPR) reduction, Maintaining Suspicious Semantics (SS) reduction, but all these methods compromise forensic analysis.

3.1.2 Generality. ProTracer[27] and Zhu et al.[46] perform fine-grained stain tracking to reduce log volume. In particular, [46] introduces data reduction based on SS strategy. It defines external nodes as suspicious nodes. When information is transmitted from suspicious nodes to other nodes, those nodes will also be tagged as suspicious, ultimately retaining only the suspicious nodes and related events. S-DPR, suggested by [20], eliminates edges based on the premise that the set of ancestors obtained during node tracing remains unchanged. Xu et al. presents the PCAR strategy[41]. It retains hot processes and related events and entities based on process activity. DistDet [8] proposes a low-consumption anomaly detection approach conducted on the host based on industrial production scenarios. It achieves data reduction by transmitting the detected anomalous nodes along with their k-hop neighboring nodes and corresponding events to the server.

These methods all reduce data based on the abnormal behavior conditions they define. This compromises the completeness of normal behavior logs. Additionally, because their definitions of abnormal behavior may not be comprehensive, it could result in pruning away abnormal logs that cannot be uploaded to the server for detection.

3.1.3 Real-time Performance. LogGC [25] proposes the use of BEEP[22] technology to identify short-lived and isolated temporary files, allowing their deletion without impacting dependency analysis and reducing storage costs. However, discovering temporary files needs events input in reverse order which is impractical in real environments and lack of real-time performance. RapSheet [15] introduces tactical graph based on the provenance graph and alarm. It significantly reduces cost of data storage by performing backward propagation on anomalous nodes to remove nodes that are not associated with the alerts. Nevertheless, since the detection is performed on the server-side, it does not satisfy the real-time performance.

3.2 Data Compression

The data compression technique does not modify the logs, so it ensures the integrity of the logs and does not effect the results of intrusion detection. However, since the detection process requires the data to be decompressed, it does not reduce the time and space overhead of intrusion detection. Data compression has been a topic of research for a long time. Graefe [11] proposes compression strategies that facilitate data queries. Current compression strategies include columnar storage in databases and compression using bitmap indexes. Sleuth [18] introduces new encoding methods and data structures to reduce storage space occupied by audit logs. SEAL [10] stores all parent nodes of a child node together and compresses edge properties to achieve query-friendly compression. DingCase [7] stores logs using node and edge tables and represents events as vectors using a DNN model and calibration table, significantly

improving compression efficiency. In [14], a finite state automaton is proposed to store event behaviors.

3.3 Intrusion Detection

The related work on intrusion detection can be mainly categorized into three types: rule-based detection, anomaly-based detection, and attack investigation.

3.3.1 Rule-based Detection. Based on prior knowledge, rule-based methods can be divided into tag propagation and rule matching. Tag propagation assigns credibility and confidentiality tags, and when highly confidential data is transmitted to untrusted nodes, alerts are generated [18, 19]. Rule matching [15, 30, 31, 39] establishes rules based on Tactics, Techniques, and Procedures (TTPs)[1], which can be a single event or a subgraph. Intrusion behaviors are discovered by matching events with predefined rules.

3.3.2 Anomaly-based Detection. Anomaly detection methods mainly utilize graph algorithms to extract node or graph features. They employ unsupervised or self-supervised methods to detect anomalies [13, 28, 33, 34, 37, 38, 42, 44, 45]. Threatrace [38] employs the GraphSAGE[12] model and utilizes a self-supervised approach with node type prediction for training. Kairos[6] further utilizes the TGN model as an encoder and the MLP model as a decoder to calculate event anomaly scores by predicting edge type. Flash[33] uses the doc2vec method to initialize node and event features. It employs and trains the GraphSAGE model in the same way as Threatrace. It then employs the XGBoost model to perform node classification tasks for anomaly detection.

3.3.3 Attack Investigation. Attack investigation is necessary for reconstructing the attack scenario. To address the problem of dependency explosions during the traceback process of Point of Interests (POI), two main approaches are commonly employed. The first approach involves supplementing system logs to reduce false dependencies during traceback[17, 24, 26, 43]. OmegaLog [17] proposes binary analysis techniques to obtain application program logs and combines them with system logs to prune false dependencies. ALchemist[43] also reduces false dependencies by extracting application program logs and integrating them into system logs to alleviate dependency explosions. If deleting duplicate edges does not change the results of forensic analysis, it will not affect the results of attack investigation in these works. Another approach directly detects attack path in the traced graph by formulating strategies for calculating anomaly scores[9, 16, 40]. DEPCOMM [40] divides the data provenance graph into multiple modules by applying a random walk method. It then employs a strategy to compute the anomaly score for each vertex, taking into account both its in-degree and out-degree. DEIMPACT[9] further calculates weights by evaluating the similarity between events and POI events based on data flow volume, timestamp, and in-degree/out-degree. It assigns anomaly scores to events using KMeans and Linear Discriminant Analysis.

4 METHOD

In this section, we present the design of AUDITRIM. The main components are shown in Figure 4 and presented as follows.

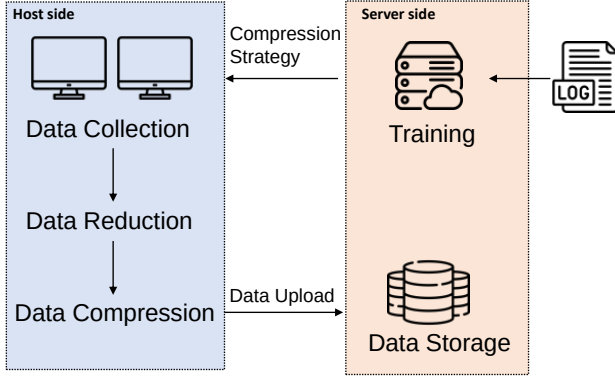


Figure 4: The overview of AudiTrim.

- **Data Reduction.** This component involves summarizing the occurrence types of redundant edges and analyzing their impact on attack investigation to perform redundant edges reduction.
- **Data Compression.** This component focuses on identifying high-frequency items and regular properties within the audit logs and employs Huffman[21] trees for symbol substitution to reduce storage space.

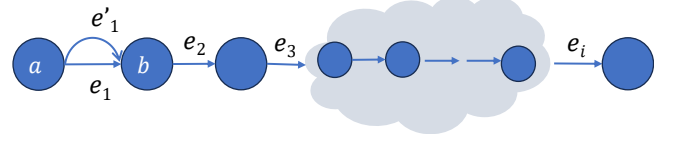
4.1 Data Reduction

4.1.1 Duplicate Edges. To ensure the generality of data reduction, we primarily constrain them from the perspectives of attack investigation and anomaly detection.

The attack investigation process aims to reconstruct the attack scenario by tracing back to the POI events. Typically, this is achieved by performing backward propagation from the POI events. Some approaches also require forward propagation from the POI events or the root vertex obtained through backward propagation to gain a better understanding of the intrusion process. In order to ensure the generality of the data reduction approach, we analyze the traceability of the redundant edges. Previous works also attempt to remove duplicate edges, but due to the lack of discussion on the conditions of duplicate edges, some types that could be pruned are overlooked, leading to inefficient data reduction.

In anomaly-based intrusion detection, benign logs are used as input, and the goal is to detect abnormal events by learning from benign behavior in the logs. In practice, a single user action can generate a large number of duplicate system call logs, which may cause the model to overlook other relevant event logs during the learning process. We propose a time window strategy based on process behavior to merge events. For example in Figure 3, by merging these logs, not only is the log volume reduced, but it also helps the model better learn user behavior.

For events $e_1(u_1, v_1, t_1)$ and $e_2(u_2, v_2, t_2)$, if they have the same source vertex, destination vertex and event type, they are referred to as duplicate events, i.e., $u_1 = u_2$ and $v_1 = v_2$. Duplicate events will generate duplicate edges in the dependency graph. For events that do not have duplicate events, deleting these events will result in the loss of logs when propagating from their causality dependent events, leading to incomplete results in the attack investigation

Figure 5: The event timestamps satisfy $t_1 < t_2 < t_3 < \dots < t_i$ and $t_2 < t'_1$.

and potential risk of missing anomalous events. Therefore, non-duplicate events should not be pruned. To ensure that the reduction of duplicate events does not impact intrusion detection, we propose causality dependency analysis and time window strategy.

4.1.2 Causality Dependency Analysis. For duplicate events e_1 and e'_1 in Figure 5, they have same source vertex a and destination vertex b . However they have different timestamps, which satisfy $t_1 < t_2 < t'_1$. Define

$$E_1 = \{e = (u, v, t) | u \neq a \wedge u \neq b \wedge v \neq a \wedge v \neq b\}.$$

E_1 is the set of edges where the edges do not directly connect with e_1 and e'_1 . Edges in E_1 may or may not have causality dependency with e_1 and e'_1 . To determine whether any edge in E_1 has causality dependency with e_1 , it is necessary to find (or cannot find) a path from this edge to e_1 . If the path is very long or the graph is large, it becomes difficult to make this determination.

As shown in Figure 5, e_i belongs to the set E_1 , but e_i have causality dependency with e_1 , without e'_1 . This is caused by e_2 . e_2 can pass information to e_i , and e_1 have direct causality dependency with e_2 , so e_1 can pass information to e_i through e_2 . However, e'_1 cannot pass information to e_2 , and therefore cannot pass information to e_i . If merging duplicate events does not affect their original direct causality dependent events, then the merged event can transmit information through its direct causality dependent events to its original causality dependent events. **If merging duplicate events does not affect the direct causality dependency, it will also not affect its causality dependency**, and thus will not impact the results of attack investigation. By examining the direct causality dependency of duplicate events, various scenarios when duplicate edges occur can be summarized.

There are two conditions for an event's direct causality dependency, corresponding to traceability and propagation. For duplicate events, considering the temporal relationship, there are four conditions for the direct causality dependency. As shown in the Figure 6, this is a temporal propagation graph that directly reflects the temporal information and dependency relationships. The vertical lines represent vertex, and due to the large number of related events for each vertex, a timeline is established. The edges above represent events that occur earlier, while the edges below represent events that occur later. Both tracing and propagating need to consider the chronological order of events, the attack investigation event can only trace along the upper-left direction or propagate along lower-right direction. In the Figure 6, e and e' are duplicate events. e_1 and e_4 have direct causality dependency with e and e' , deleting event e or e' will not change the semantics of event e_1 and e_4 , and it will not affect the results of the attack investigation. For e_2 , e_2 has direct causality dependency with e' not e , deleting e' will result in a loss of semantics for e_2 , and propagating e_2 would lead to the inability

to propagate to e_4 correctly. Nevertheless, since e is unrelated to events e_2 , deleting e will not result in a loss of semantics for e_2 . Regarding e_3 , it has direct causality dependency with e but not e' . Therefore, deleting e will affect the backward trace results for e_3 , while deleting event e' will not have an impact on the results.

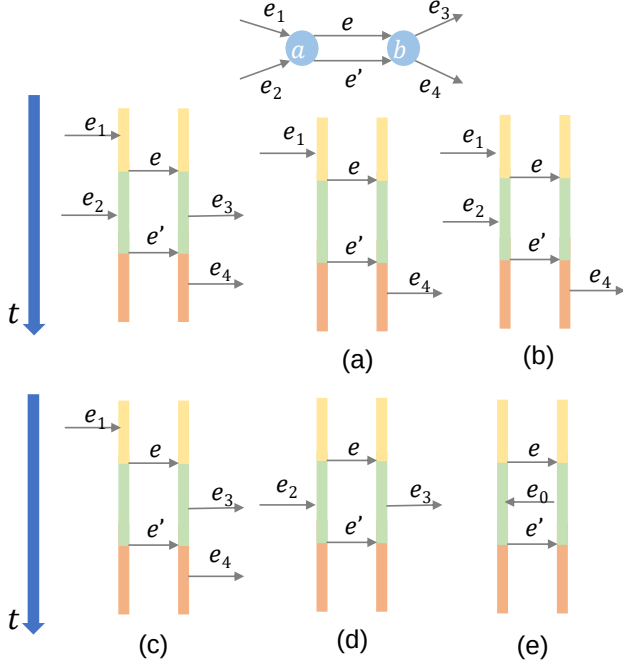


Figure 6: Temporal propagation graph. According to the timeline, the edge above occurs earlier. Causality analysis can only trace along the upper-left direction or propagate along the lower-right direction.

As shown in Figure 6, the occurrence of duplicate edges can be classified into the following types.

- (1) In Figure 6(a), e and e' have a common direct causality dependency, deleting e or e' will not affect the causality analysis.
- (2) In Figure 6(b), deleting e will not affect the causal analysis while deleting e' will change the semantics. As shown in Figure 7(a), P_2 reads the content of file F , and then P_1 modifies F , and then P_2 reads F again. P_2 can receive the information written by P_1 , and deleting the event r_1 and r_2 will not affect the information propagation and the semantics of the write event.
- (3) In Figure 6(c), deleting e' will not affect the causal analysis. As shown in Figure 7(b), P_2 firstly writes to F , P_1 reads from F , and then P_2 writes to F again. Deleting the event w_3 and w_4 will not affect the flow of information from P_1 to P_2 , and it will not affect the forensic analysis results.
- (4) In Figure 6(d), e and e' are both necessary in attack investigation, there are no redundant events can be removed.
- (5) In Figure 6(e), the presence of cycles in the dependency graph leads to this duplicate events. As shown in Figure 8, when a process alternates between reading and writing to

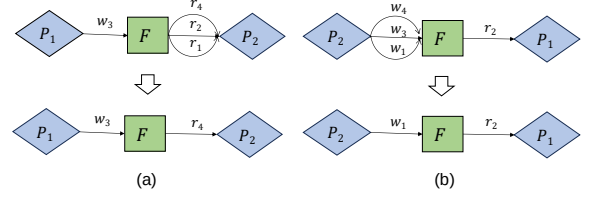


Figure 7: P_2 continuously reads(writes) from the file, while P_1 writes(reads) to the file during this period.

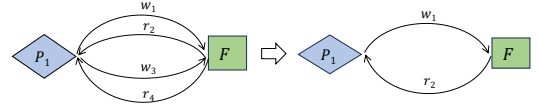


Figure 8: Process reads and writes to file alternatively.

a file, every read operation and every write operation have causality dependency, so keeping one read and write event can ensure that there is no impact on the causality analysis.

4.1.3 Time Window Strategy. By employing the causality dependency analysis, it is ensured that the reduction of duplicate events does not alter the results of traceability and propagation. However, in attack investigation, causality dependency is only part of it, and due to the occurrence of dependency explosion in the traceability processes, it imposes an unbearable workload on operational personnel. Therefore, there are many works that process the traceability results. In these works, calculating anomaly scores based on expert knowledge is a common approach. Event frequency, time intervals, and data size are the main factors used to calculate anomaly scores[16][9]. Consequently, we need to ensure that data reduction will not have a negative impact on these features. Besides, in anomaly-based intrusion detection, benign logs are used as input, and the goal is to detect abnormal events by learning from benign behavior in the logs. We must guarantee the integrity of user behavior in the logs to ensure the efficiency of anomaly detection. Therefore, after causality dependency analysis, we further propose the time window strategy to constrain the reduction of duplicate events.

Regarding the analysis of duplicate events, there are mainly two scenarios for the occurrence of duplicate events. One type is duplicate events generated by multiple behaviors, and the other type is multiple duplicate events generated by a single behavior. For the first type of duplicate events, they can reflect user behavioral characteristics. For example, users may frequently use certain applications or browse specific websites, leading to a high frequency of certain events. Merging these events would not only affect the frequency features but also compromises the completeness of user behavior. In contrast, for the second type of duplicate events, they can negatively affect feature extraction. As illustrated in Figure 9, in the DARPA E3 dataset, when a host downloads a file from the network, a large number of EVENT_RECV logs are generated in the logs due to continuous data transmission. A single file download behavior

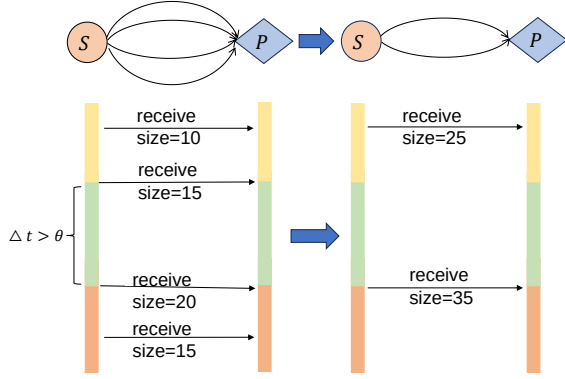


Figure 9: An example for time window. Δt is the time interval of two events. θ is the threshold for time window.

is incorrectly counted as thousands of downloads events, and the original file size is dispersed across multiple EVENT_RECV events. This is unreasonable. Merging these duplicate events, on the other hand, provides a more accurate representation of user behavior in terms of frequency, data size, and other features. It is also more suitable for the training and testing processes of anomaly-based intrusion detection models. Therefore, distinguishing between these two cases of duplicate events is crucial.

The first type of duplicate events is generated by different behaviors, and in most cases, these duplicate events have distinct timestamps. The second type of duplicate events, generated by a single action, typically has very small time intervals between them. Therefore, we can merge the second scenario of duplicate events by setting a time window to ensure integrity. For the first category of repeated events, the time difference is on the millisecond level, and initially, we define a time window threshold of 1 second. When repeated events e and e' meet the reduction criteria, we compare the time difference between the two events' timestamps. If it is less than the threshold, we perform reduction. Furthermore, since the initial threshold may not be suitable for all cases, different types of duplicate events may correspond to different thresholds. Therefore, adopting a dynamic adjustment of the time window size after each pruning operation. We calculate $threshold_e = \min(1, 2 * (t_{e'} - t_e) + 0.001)$ as the threshold for event e . Considering the fluctuation in the time intervals of the first category of repeated events, which are much smaller than those of the second category, we choose to use $2 * (t_{e'} - t_e)$ instead of $t_{e'} - t_e$. This adjustment helps to increase the threshold effectively, enabling more comprehensive reduction of the first category of repeated events. Additionally, since sometimes two repeated events may have identical timestamps, we add 0.001 to prevent the threshold from being zero.

When a redundant event e'' of e occurs, we compare $t_{e''} - t_{e'}$ with $threshold_e$ to decide whether pruning should be performed and recalculate the $threshold_e$.

The time window strategy not only ensures the integrity but also guarantees the real-time efficiency and reduces the memory overhead of the data reduction process. Without the time window strategy, as the number of processed events increases over time,

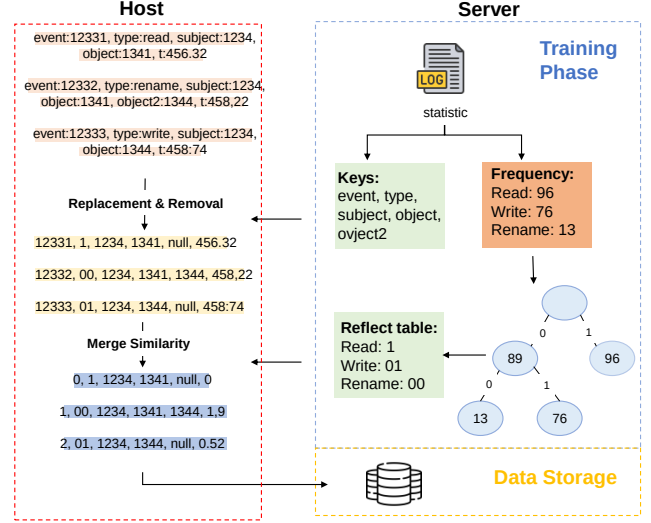


Figure 10: An example for data compression.

the need to maintain more vertices would consume huge memory resources. However, with the incorporation of the time window strategy, we only need to maintain the event information within a specific size, and when an event is outside the time window, the memory it occupies can be promptly released. Therefore, the size of the time window determines the required memory for AUDITRIM to operate.

4.2 Data Compression

To ensure data integrity, only redundant edges are pruned. During the data reduction process, causality dependency analysis and the time window strategy are incorporated to further restrict the process. Consequently, the proportion of redundant edges determines the upper limit of reduction efficiency. In the DARPA TC dataset, even with the merging of all duplicate edges, the compaction ratio can only reach up to 5x-10x. To realize higher compaction efficiency, reduction and compression tasks can be combined. Since the data reduction method does not disrupt the data structure and data compression does not compromise the integrity of the logs, there is no need to worry about the compatibility between the reduction and compression methods, nor the impact of compression on the intrusion detection process. By compressing the pruned logs, efficient data storage can be achieved, significantly reducing the bandwidth required for uploads on hosts and the data storage overhead on server.

Existing compression methods can already achieve efficient compression, but when selecting a compression method, it is important to consider not only efficiency but also real-time performance and low overhead. In this section, a lightweight compression strategy based on host logs is proposed. This compression method is not presented as a main contribution of this paper.

As shown in Figure 10, data compression consists of two components. Firstly, on the server-side, high-frequency fields such as event types and node types are identified by analyzing a training set of host logs collected over a period of 7 days or a month. These

fields are then mapped to bit encoding using Huffman Tree. Additionally, since log information is commonly presented in key-value format, specifying the position of the keys in advance can save bandwidth by reducing the need to upload key fields. The server uploads the encoding rules to user, where the data are encoded and reconstructed based on these rules.

The second component is performed on the host. A vertex table is established to store vertex identifiers and attributes. Since event identifiers occur only once, mapping of event identifiers is not required. The vertex table simplifies the content related to vertex in the audit logs. During upload, both the vertex table and the compressed logs are sent to the server.

5 EVALUATION

In this section, we perform evaluation on AUDITRIM. We conducted tests on AUDITRIM using three publicly available datasets to evaluate its integrity, real-time performance, low resource consumption, and efficiency. The evaluation was centered around the following research questions:

RQ1: How much does the data reduction method improve reduction efficiency compared to existing approaches?

RQ2: Can the adopted data compaction method outperform state-of-art compaction techniques to achieve high-efficiency compaction?

RQ3: How does the AUDITRIM perform in terms of memory usage and time overhead? Does it impact the user experience?

RQ4: Does the pruned data affect the result of intrusion detection algorithm? Can it enhance intrusion detection efficiency?

The evaluation section comprises four main parts: baseline, dataset introduction, efficiency experiments, and integrity verification. The datasets used include DARPA E3, DARPA E5, DARPA OPTC, CrackHost, and PublicArena. In the efficiency experiments, we compared the results with CPR[41], GS[46] and F-DPR[20] as the baselines to validate the data reduction algorithm's efficiency across the four datasets. For integrity verification, we employed DEPIMPACT[9] and NODOZE[16] algorithms, using their source code to conduct attack investigation experiments on the CrackHost dataset. Additionally, we employed Threatrace[38], Kairos[6] algorithms for intrusion detection experiments. We confirmed that pruned data is beneficial for anomaly-based intrusion detection.

Our results demonstrate that the AUDITRIM outperforms existing approaches in data reduction (RQ1) while consuming minimal memory (RQ3). Furthermore, it achieves higher compaction efficiency (RQ2), enabling real-time data compaction. Importantly, the pruned data retains its integrity, ensuring the efficiency of attack investigation and anomaly-based intrusion detection while significantly reducing time and memory overhead (RQ4).

5.1 Baseline

Inam et al.[23] propose a comprehensive work on intrusion detection. It provides a detailed overview of data reduction methods and conducts experiments using these approaches, including LogGC[25], F-DPR[20], CPR[41], PCAR[41], LogApprox[29], S-DPR[20], NodeMerge[35] and Induction[14]. These data reduction methods are also extensively introduced in Section 3.1. Among these, Loggc lacks real-time performance and has low reduction

efficiency, making it unsuitable for deployment, so it is not considered as a baseline. PCAR, SS and S-DPR compromise data integrity, rendering the pruned data unsuitable for intrusion detection tasks, and thus lack generality. Therefore, they are not included as comparison methods. Nodemerge, Logapprox, and Induction modified the log format, and uploading them to the server requires a "decompression" step, which aligns more with compression strategies. However, their applicability is limited so they have lower compression efficiency, as shown in [23]. Therefore, they are not considered as baselines for comparison. CPR, F-DPR, and GS are all based on pruning duplicate edges and use causality dependency analysis to constrain the data reduction. They align better with the requirements of generality for our work, and hence are selected as the baselines.

5.2 Dataset

We evaluate compaction efficiency using three public datasets: DARPA, CrackHost, and PublicArena. Since they correspond to different scenarios, various datasets were selected for integrity validation and runtime performance testing, as shown in Table 1.

Table 1: Dataset Usage Scenarios

Dataset	Attack Investigation	Anomaly Detection	Runtime Performance
DARPA	×	✓	✓
CrackHost	✓	×	×
PublicArena	×	×	✓

DARPA E3 and E5 were released by DARPA Transparent Computing (TC) Program Engagements 3 and Engagements 5, conducted in 2018 and 2019. DARPA OpTC was released by DARPA Operationally Transparent Cyber (OpTC) program, conducted in 2019. Multiple teams participated, responsible for collecting audit data from different platforms such as Linux, Windows, and BSD. The DARPA dataset is currently the most common dataset used in intrusion detection and attack investigation works based on provenance graph. We conducted experiments using its sub-datasets CADETS, THEIA, and TRACE, as shown in Table 2. TRACE and THEIA were captured on the Ubuntu system, while CADETS was captured through the Causal, Adaptive, Distributed, and Efficient Tracing System (CADETS) on FreeBSD. The DARPA dataset captures logs at the system call level, providing more comprehensive information, and entities in the logs are more finely divided. Additionally, as DARPA is the most commonly used dataset for anomaly detection, we used this dataset not only for compaction efficiency and runtime performance experiments but also to validate the generality of pruned data in anomaly detection. However, since DARPA do not provide POI, it is not used for attack investigation.

CrackHost dataset is a publicly available dataset provided by the DEPIMPACT work. It was generated specifically for attack investigation tasks, which do not require training benign behaviors. As a result, compared to the other two public datasets, it has removed a large number of irrelevant benign events, making it smaller in scale. As show in Table 3, it contains only about 500,000 events and is only 62MB in size. Furthermore, unlike intrusion detection

datasets, it provides POI events for traceability, making it more suitable for attack investigation works. We not only conducted compaction efficiency experiments on this dataset but also used it to verify the generality of pruned data in attack investigation. However, due to the dataset removing a large number of irrelevant benign events, being incomplete and too small in scale, it is not used for anomaly detection and runtime performance experiments.

PublicArena dataset is provided by the DISTDET[8] work and is designed for intrusion detection tasks, similar to the DARPA TC dataset. It includes two sub-datasets (Table 4) HostA-Lateral and HostB-Insider. Due to the wide variety of attack behaviors, anomaly-based intrusion detection methods typically adopt unsupervised approaches. Each sub-dataset in PublicArena consists of three JSON files containing benign data for training, benign data for validation, and attack data for testing. The PublicArena dataset comprises over 15 million events. Compared to the DARPA dataset, it has fewer event types and simpler information, resulting in smaller storage space. Additionally, its lack of events for nodes leads to nodes not having unique identifiers, often using cmdline, file paths, and other information to differentiate nodes. This causes the same process to correspond to the same node in the graph even after being closed and run again, leading to a higher proportion of duplicate edges in this dataset and thus higher data reduction efficiency. Due to the lack of unique identifiers for nodes in this dataset and the source code from related work is not applicable to this dataset. We only evaluated compaction efficiency and runtime performance on this dataset.

Table 2: Overview of DARPA E3 and DARPA E5 dataset.

Scene	System	Edges (millions)	Size (GB)
E3-THEIA	Ubuntu	103	47
E3-TRACE	Ubuntu	31	20
E3-CADETS	FreeBSD	12	11
E5-CADETS	FreeBSD	57	54
E5-TRACE	Ubuntu	72	56
OpTC	Windows	76	40

Table 3: Overview of CrackHost dataset.

Scene	Benign Edges	Anomaly Edges	Anomaly Nodes	Size (MB)
CrackHost	484180	43	22	62.5

Table 4: Overview of PublicArena dataset.

Scene	System	Benign Edges	Anomaly Edges	Size (GB)
HostA-Lateral	Win10	6501832	1062	3.15
HostB-Insider	Winserver	9070293	1423	4.16

5.3 Effectiveness

To facilitate comparison between different approaches, the experimental process is divided into three phases. Phase one involves data reduction. In the second phase, a time window strategy is incorporated during the reduction process. Finally, the results from phase two are subjected to data compression. In phase one, a data reduction strategy without a time window is adopted for comparison with previous works. Since existing reduction methods are primarily focused on forensic analysis, and our reduction method also considers causal analysis, comparing the efficiency without a time window allows for an effective assessment of the proposed reduction method. In the second experiment, the time window strategy is introduced during the data reduction process. Subsequent integrity (attack investigation and anomaly detection) experiments are conducted to validate the generality of the result of this experiment. This part represents a further consideration of integrity compared to existing data reduction methods. In the third phase, the reduction and compression strategies are combined to form a efficient data compaction method that can be deployed in real production environments, greatly improving the data storage efficiency.

Table 5 presents the compaction ratio of AUDITRIM and baselines. The compaction ratio calculation formula is as follows:

$$\text{Compaction Ratio} = \frac{\text{Original Data Size}}{\text{Compressed Data Size}}$$

In the Table 5, CPR, GS, and F-DPR are baselines. CDA stands for our data reduction method only based on causality dependency analysis, used for comparison with baselines. TW represents time window strategy. DC stands for data compaction, including data reduction and data compression method. In the data reduction process based on causality dependency, previous works lacked exploration of scenarios with duplicate edges. DPR[41] only removes duplicate edges that satisfy scenarios in Figure 6(a). GS[46] and F-DPR[20] perform data reduction on duplicate edges that satisfy Figure 6(a) and (c) scenarios. In contrast, we conducted discussions and analysis to perform data reduction on more scenarios. CDA considered a wider range of conditions where duplicate edges can be pruned, resulting in approximately 2× improvement in data reduction efficiency compared to existing methods. In the CDA+TW column, to ensure the integrity of the audit logs, the time window constraint was further added. It reduced the data reduction efficiency but still achieved a compaction ratio of 3.0-8.7×. This efficiency is close to the proportion of duplicate edges in the dataset. In Table 5, the DC column, which combines reduction and compression methods, achieves a compaction ratio of 26-65×, meeting efficiency requirements. Furthermore, the compression methods can be replaced with more efficient compression methods to achieve even higher efficiency.

Analyzing the efficiency of the reduction strategy, we observed that due to the different granularity of data collection, the reduction efficiency is much higher in the PublicAren dataset compared to the DARPA dataset. However, after introducing the time window strategy, the reduction efficiency in the PublicAren dataset significantly decreases, while for the DARPA dataset, the reduction rate only experiences a minor decrease. This difference is attributed to the use of different vertex identifiers in the two datasets. In the PublicAren dataset, fields such as PNAME and FILEPATH are used as vertex identifiers, whereas in the DARPA dataset, a combination

Table 5: The compaction ratio of AUDITRIM and baselines on datasets. * represents the baseline. CDA is causality dependency analysis, used for comparison with baselines. TW represents time window strategy. DC stands for data compaction, including data reduction and data compression method.

Dataset	CPR*	GS*	F-DPR*	CDA	CDA+TW	DC
E3-THEIA	2.7×	3.5×	4.1×	7.8×	7.2×	65×
E3-CADETS	1.4×	1.8×	2.1×	5.4×	4.8×	42×
E3-TRACE	1.8×	2.4×	2.6×	6.2×	5.7×	52×
E5-CADETS	1.9×	2.2×	3.1×	7.1×	6.3×	56×
E5-TRACE	1.9×	2.1×	2.5×	3.2×	3.0×	26×
OpTC	2.6×	2.8×	3.1×	4.1×	3.7×	32×
PublicArena	12.4×	17.8×	21.4×	32.2×	8.7×	63×
Crackhost	3.2×	5.0×	5.8×	8.7×	8.2×	54×

of timestamp, host number, and pid is used as the process identifier. For example, when a process terminates and then restarts, it will be treated as the same vertex in the PublicArena dataset, while in the DARPA dataset, it will be considered a new vertex. Without the time window strategy, events in the PublicArena dataset occurring within the time window will be merged, whereas in the DARPA dataset, they will remain independent.

Table 6 shows the reduction efficiency on different types of events in the E3-CADETS dataset. By analyzing the data reduction results of different event types, we can observe the occurrence of duplicate edges for each type of event. EVENT_MMP and EVENT_LSEEK events have a high occurrence rate and are almost entirely pruned in the DPR[41] works, indicating that these events often occur in sequence. EVENT_MMP is used for memory allocation, and when opening a file, this event is continuously called to allocate enough space. EVENT_LSEEK, as a system call for changing the file pointer position, is continuously called during file writing and reading, generating a large number of consecutive event logs. EVENT_EXECUTE, EVENT_FORK, EVENT_CONNECT, and other events have a low occurrence rate and are almost not pruned. This is because these system calls often lead to the generation of new vertices, and the edges associated with them are first-time occurrences, so they are not pruned. The reduction ratio for EVENT_READ events is similar in the DPR, GS and F-DPR works since, in most cases, EVENT_READ events are related to EVENT_WRITE events. For example, during employee file inspection, if issues are found, the file may be modified, generating Figure 6(b) duplicate edges. Therefore, in AUDITRIM, the reduction efficiency for this system call has been significantly improved. By analyzing these statistical data can help researchers to understand the process of these system calls and also provides insights into the types of duplicate edges that may be generated for these events.

5.4 Runtime Performance of AUDITRIM

In this section, we mainly focus on evaluating the runtime performance. We conducted experiments to assess the memory usage, processing speed, and CPU utilization during the AUDITRIM runtime. Since the CrackHost dataset is relatively small, we conducted experiments on the DARPA and PublicArena datasets. The experiments are conducted separately on two machines: one with 64

vCPUs and 128GB of RAM running Ubuntu 16.0.1, representing the experimental environment, and another with 16 vCPUs and 16GB of RAM running Windows 10, simulating an employee’s working environment.

As shown in Figure 11(a), even though the E3-CADETS dataset is twice the size of the HostA-Lateral dataset, the memory usage during the experiments is comparable. As mentioned earlier in Section 4.1.3, each event is stored in the user’s host for a time window before being uploaded. Consequently, the data compaction process only stores audit logs of a time window’s size. Therefore, during the running process on Windows 10, the average memory usage was less than 1%, which do not impact employees’ daily work and activities.

In terms of data processing efficiency, as shown in Figure 11(c) and 11(d), on the server, the 11GB CADETS dataset was processed in just 154 seconds, processing 70MB of data per second. On the personal computer, it only took 360 seconds to process, achieving a speed of processing 30MB of data per second. Assuming hosts of active users can generate 5GB of audit logs daily, and this data is generated within 10 hours, the log generation rate would be 0.14MB per second, which is far below our data processing rate. The efficient data processing speed ensures that our data compaction process can be performed in real-time, as well as enabling real-time data uploading and server-side intrusion detection.

Figure 11(b) shows the average CPU utilization. When processing the CADETS data, the server’s CPU utilization was only 1.6%, and on the user’s host, it required approximately 4.6% CPU utilization. Moreover, since the experiments were conducted on public datasets where data processing is more concentrated, in a real-world scenario, data would be captured and processed in parallel. Consequently, the amount of data to be processed within a specific time would be only about 0.5% of the experimental setup, resulting in significantly lower CPU utilization.

We also measured the runtime performance of the baselines. Table 7 shows the runtime performance results of baselines applied to the CADETS dataset on the Windows machine. AUDITRIM’s time consumption and CPU utilization are slightly higher, but due to the addition of the time window strategy, AUDITRIM uses significantly less memory compared to existing methods.

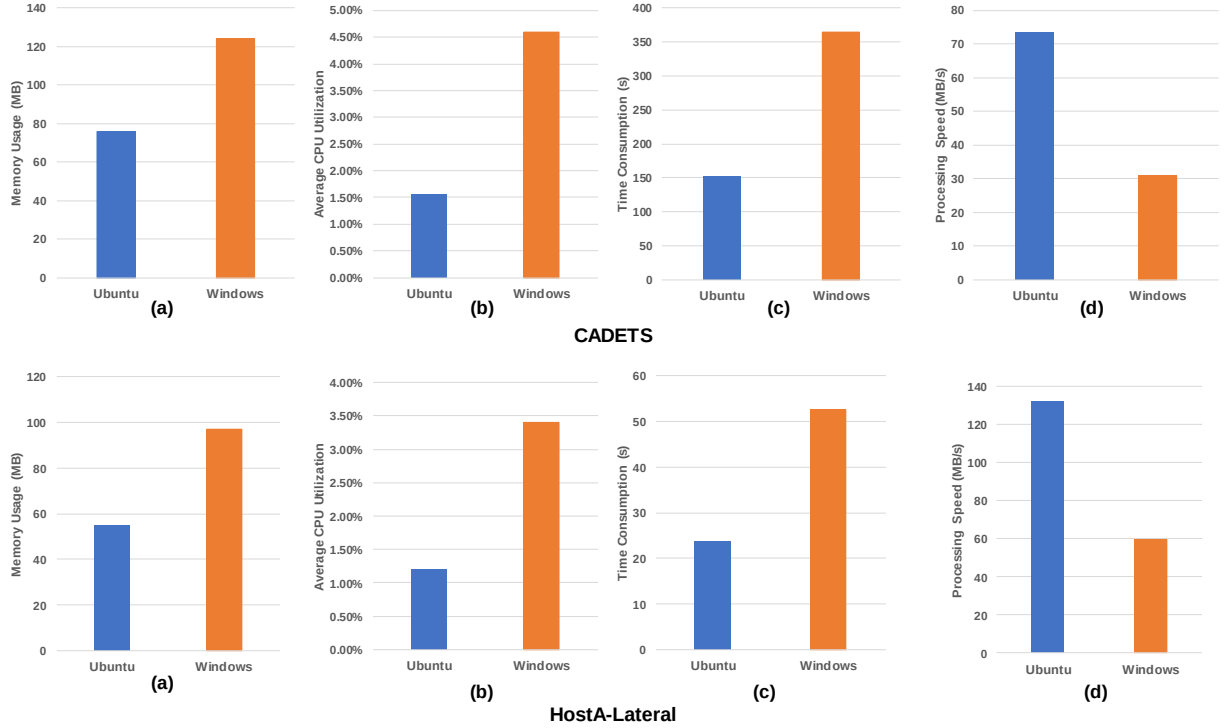
Based on the experimental results, we can conclude that AUDITRIM has excellent data processing speed, meeting real-time requirements in practical environments. Additionally, both memory usage and CPU utilization are low, ensuring minimal burden on user hosts and fulfilling the low-energy consumption requirements in industrial environments.

5.5 Generality

To further demonstrate the generality of the reduction method, we conducted integrity analysis on data reduction and tested for forensic analysis. Subsequently, we performed experiments using existing works in both attack investigation and anomaly-based intrusion detection. We tested not only the generality of the AUDITRIM method but also the CPR and CDA. CPR is based on causality dependency for duplicate edges reduction, and the events it prunes are a subset of other baselines. Therefore, its reduction is less aggressive,

Table 6: Efficiency of Reduction for Different Events

Algorithm	TOTAL		CPR		GS		F-DPR		AudiTrim	
TYPES	Num	Propotion	Reduce	Impro	Reduce	Impro	Reduce	Impro	Reduce	Impro
EVENT_READ	3941401	34.45%	54.1%	2.18×	55.6%	2.25×	64.2%	2.79×	90.4%	10.42×
EVENT_CLOSE	1993972	17.43%	36.5%	1.57×	36.9%	1.58×	37.6%	1.60×	78.4%	4.63×
EVENT_MMP	1479028	12.93%	99.99%	10000×	99.99%	10000×	99.99%	10000×	99.99%	10000×
EVENT_LSEEK	1274837	11.14%	99.99%	10000×	99.99%	10000×	99.99%	10000×	99.99%	10000×
EVENT_OPEN	1225765	10.71%	6.0%	1.06×	6.1%	1.06×	14.2%	1.17×	28.6%	1.40×
EVENT_WRITE	394042	3.44%	45.8%	1.84×	57.2%	2.34×	73.6%	3.79×	89.4%	9.43×
EVENT_FORK	73850	0.65%	0.0%	1.0×	0.0%	1.0×	0.0%	1.0×	0.0%	1.0×
EVENT_EXECUTE	70497	0.62%	0.0%	1.0×	0.0%	1.0×	0.0%	1.0×	0.0%	1.0×
EVENT_RECVFROM	32432	0.28%	25.8%	1.3×	63.2%	2.72×	65.7%	2.91×	66.2%	2.96×
EVENT_SENDTO	30550	0.27%	60.7%	2.5×	63.7%	2.75×	67.2%	3.05×	90.9%	10.99×
EVENT_CONNECT	28273	0.25%	0.0%	1.0×	0.0%	1.0×	0.0%	1.0×	8.2%	1.09×
EVENT_RECVMSG	1635	0.014%	26.50%	1.7×	26.80%	1.37×	31.30%	1.45×	73.20%	3.73×
EVENT_SENDMSG	898	0.008%	48.90%	2.0×	48.90%	1.96×	49.60%	1.98×	52.10%	2.09×

**Figure 11: The runtime performance of AudiTrim.**

ensuring better generality. The generality evaluation of CPR can assess the generality of data reduction based on causality dependency. CDA is our reduction method but without time window strategy, by comparing CDA with AudiTrim, it can be demonstrated that the time window strategy enhances generality, thus proving the necessity of the time window strategy.

5.5.1 Integrity of Anomaly Data. For integrity analysis, we employed the DARPA E3 and firstly identified all anomalous events in the complete data. We then searched for these events in the pruned data, and all anomalous events are present in the pruned data, confirming that our data reduction method preserves the integrity of anomalous data. To verify that data reduction does not affect forensic analysis, we conducted a traceback on the POI nodes in the

Table 7: Runtime performance of AudiTrim and baselines.

Method	Time Consumption (s)	CPU Utilization	Memory Usage (MB)
AudiTrim	382	4.50%	120
F-DPR	357	4.20%	520
GS	363	4.20%	460
CPR	278	3.80%	380

CrackHost dataset. As shown in Table 8, Complete indicates that the data is complete. AudiTrim indicates that the data is pruned by AudiTrim. CPR indicates that the data is pruned by CPR. CDA is our reduction method but without time window strategy. In the row of BACKTRACK, the count of false negatives is zero in AudiTrim, CPR and CDA rows, indicating that reduction with these methods does not result in the loss of attack events. Moreover, the pruned traceback graph, including AudiTrim, CPR and CDA, has significantly fewer edges compared to the complete traceback graph, mitigating the dependency explosion phenomenon and improving traceback efficiency.

Table 8: Attack Investigation on CrackHost. Complete indicates that the data is complete. AudiTrim indicates that the data is pruned by AudiTrim. CPR indicates that the data is pruned by CPR. CDA represents causality dependency analysis. It is AudiTrim without time window.

ALGORITHM	DATA	FP	FN	Edges
NODOZE	Complete	1463	0	1506
	AudiTrim	1460	0	1503
	CPR	1463	0	1506
	CDA	1460	0	1503
DEPIMPACT	Complete	621	0	664
	AudiTrim	621	0	664
	CPR	621	0	664
	CDA	621	0	664
BAKCTRACK	Complete	11016	0	11059
	AudiTrim	2960	0	3003
	CPR	4126	0	4169
	CDA	2653	0	2696

5.5.2 Attack Investigation. To assess attack investigation efficiency, we selected NODOZE[16] and DEPIMPACT[9] as representative works to show that reduction does not reduce the efficiency of attack investigation. We used the CrackHost dataset provided by DEPIMPACT, and their source code and runnable programs allowed direct experimentation on this dataset. For DEPIMPACT, which calculates path anomaly scores based on attributes like in-degree, out-degree, and timestamps, both pruned and complete experiments yielded identical path anomaly scores and entry points, as shown in Table 8. The resulting attack traceback graphs, with equal false positives and false negatives, were also the same. As for NODOZE, which utilizes frequency-based methods to calculate anomaly scores, there were minor differences in the computed path anomaly scores. However,

the FP, FN, and final pruned graph size remained consistent, further confirming that merging duplicate edges has minimal impact on frequency-based statistics. These results indicate that AudiTrim ensures the generality of attack investigation. CPR and CDA also do not affect the results of attack investigation. It proves that the causality dependency analysis can ensure the accuracy of attack investigation without the need for the time window strategy. Notably, the attack traceback graph on pruned logs exhibited a smaller and more precise size. Furthermore, leading to a significant reduction in attack investigation time. Although similar iteration counts during loss convergence in both pruned and complete data, the attack investigation on pruned data led to a 70% reduction in total runtime and an approximately 60% decrease in memory usage. These experiments demonstrated that our data reduction algorithm does not affect attack investigation results, moreover, enhances its efficiency.

5.5.3 Anomaly-based Intrusion Detection. To assess the impact on anomaly-based intrusion detection results, we selected Threatrace[38], and Kairos[6] as representative works. Threatrace uses graph neural networks (GNN) for anomaly detection. Kairos uses TGN and MLP as encoder and decoder to detect anomaly edges.

Table 9: Results of Threatrace. Complete indicates that the data is complete. AudiTrim indicates that the data is pruned by AudiTrim. CPR indicates that the data is pruned by CPR. CDA represents causality dependency analysis. It is AudiTrim without time window.

E3-THEIA	Precision	Recall	Accuracy	F-score
Complete	0.4	0.998	0.992	0.570
AudiTrim	0.81	0.998	0.993	0.894
CPR	0.32	0.998	0.992	0.485
CDA	0.36	0.998	0.992	0.528

As shown in Table 9, the Threatrace model exhibits significantly better detection performance on data pruned by AudiTrim compared to complete data. This is because pruned data more accurately reflects the event scenarios in different situations. For instance, in the case of file modification operations, multiple consecutive or alternating read and write events can be condensed into two simple log entries after reduction. This aids in learning behavioral patterns for the model and facilitates understanding for operational personnel. However, detection results on data pruned by CPR and CDA declined, indicating that pruning duplicate edges based solely on causality dependency analysis does not ensure intrusion detection efficiency. Adding the time window improves intrusion detection efficiency, ensuring the generality of data reduction. Therefore, the time window strategy is essential.

Regarding the Kairos experiment results, as shown in Table 10 these results are evaluated at the graph level on E3-CADETS. Although it may seem that accuracy decreases with pruned data, this is not an accurate judgment criterion. Data reduction reduces the total number of time windows and decreases the size of each graph. As a result, in practice, operational personnel need to review fewer logs when inspecting alarms on pruned data. Similarly, detection results on data pruned using CPR and CDA also declined, confirming

Table 10: Result of Kairos.

E3-CADETS	FP	FN	TP	TN	Precision	Recall
Complete	1	1	3	174	0.75	0.75
AudiTrim	1	1	3	154	0.75	0.75
CPR	5	1	3	168	0.375	0.75
CDA	3	4	0	153	0	0

that the time window strategy is necessary to ensure the generality of data reduction.

5.6 Influence of Threshold

Table 11: The influence of time window size. Dynamic indicates that the time window size automatically adjusts. Fixed means the time window size is unchangeable and set to the threshold.

Time Window Size (s)	0.01	0.1	1	10	100
Dynamic	6.7×	7×	7.2×	7.4×	7.5×
Fixed	6.7×	7.1×	7.3×	7.6×	7.8×

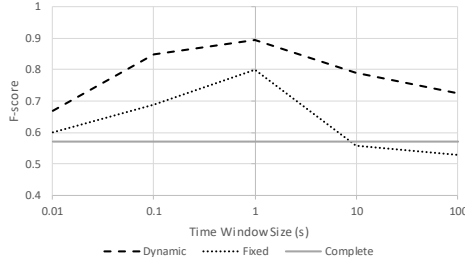


Figure 12: The influence of threshold on F-score of Threatrace.

To ensure the generality of data reduction, we proposed the time window strategy. Table 11 and Figure 12 show the compaction ratio and F-score of Threatrace on THEIA dataset with initial time window sizes set to 0.01s, 0.1s, 1s, 10s, and 100s. We also measured the impact of threshold selection on both fixed time window size and dynamic time window size. It can be observed that the reduction efficiency and generality on dynamic group are less affected by threshold selection compared to the fixed group. Moreover, setting the time window size to 1 second ensures better efficiency and generality.

6 DISCUSSION

6.1 Efficient Compression Methods

For data compression strategy, while we have improved data compression efficiency through log data analysis, we have chosen not to further compress the data to facilitate querying. Although combining data reduction strategy can achieve high compaction rates,

there is still room for improvement in the compression methods. Additionally, for reduction operations, since the proportion of duplicate edges in the log is limited, the efficiency of reduction strategies is bounded while maintaining causality dependencies. Compression methods do not compromise the integrity of logs and can be combined with reduction methods. AUDITRIM employs a simple and convenient compression strategy. In the related work, more effective methods are mentioned, and there is extensive research in the field of data storage and databases. However, when choosing compression methods, the efficiency of compression and decompression operations must be considered, as frequent queries will be performed during intrusion detection.

6.2 Compaction on Prior Knowledge

For data reduction work, the efficiency of data reduction strategy has a limit due to the limited proportion of duplicate edges in the logs without compromising forensic analysis. Currently, data reduction methods rely on prior knowledge to formulate rules for identifying log entries irrelevant to anomalies, such as using taint propagation or removing read-only files and system processes. While these methods can effectively enhance reduction efficiency, they still face two main challenges: 1) Deployment Difficulty with Prior Knowledge: Incorporating comprehensive prior knowledge into enterprise deployment is challenging. Furthermore, the cost of defining initial tainted nodes on the dataset is entirely different from the real-world deployment, which would require ongoing maintenance by employees. 2) Identifying Benign Log Entries: It is challenging to determine whether the deleted "benign" logs contain anomalies. Even with sufficient expert knowledge, many APT organizations utilize zero-day vulnerabilities to launch attacks, meaning that what we consider trustworthy processes may be exploited for intrusion. If we upload these logs to the server, subsequent server-side behavior analysis may uncover anomalies. However, once these logs are reduced from the host, the server will not detect and prevent intrusion due to the absence of anomalous logs. Additionally, it will be unable to conduct attack investigations to reconstruct the attack scenario. Addressing these challenges effectively, utilizing prior knowledge to delete real benign events, is a crucial area for future data reduction research.

7 CONCLUSION

This paper presents the data compaction system AUDITRIM, which is designed for real-world environments. By studying the conditions of duplicate edges and conducting surveys on attack investigation and anomaly-based intrusion detection works, we propose an efficient data reduction algorithm. We incorporate a time window strategy to ensure low memory and CPU usage during the reduction process, enabling real-time data reduction and maintaining its generality. Additionally, we integrate data compression algorithm, achieving compaction rates of 26×–65×, significantly reducing the bandwidth overhead of log uploads. The result demonstrates that AUDITRIM possesses four key features: low overhead, real-time capability, generality, and high efficiency, meeting the practical demands of industrial environments.

REFERENCES

- [1] 2020. MITRE ATT&CK. <https://attack.mitre.org/>.
- [2] 2021. Event Tracing for Windows (ETW). <https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/event-tracing-for-windows-etw->.
- [3] 2021. Linux Kernel Audit Subsystem. <https://github.com/linux-audit/audit-kernel>.
- [4] Mathieu Barre, Ashish Gehani, and Vinod Yegneswaran. 2019. Mining Data Provenance to Detect Advanced Persistent Threats. In *11th International Workshop on Theory and Practice of Provenance (TaPP 2019)*. USENIX Association, Philadelphia, PA. <https://www.usenix.org/conference/tapp2019/presentation/barre>
- [5] Ghita Berrada and James Cheney. 2019. Aggregating unsupervised provenance anomaly detectors. In *11th International Workshop on Theory and Practice of Provenance (TaPP 2019)*. USENIX Association, Philadelphia, PA. <https://www.usenix.org/conference/tapp2019/presentation/berrada>
- [6] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2023. Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance. *arXiv preprint arXiv:2308.05034* (2023).
- [7] Hailun Ding, Juan Zhai, Dong Deng, and Shiqing Ma. 2023. The Case for Learned Provenance Graph Storage Systems. USENIX Association. <https://www.usenix.org/conference/usenixsecurity23/presentation/ding-hailun-provenance>
- [8] Feng Dong, Liu Wang, Xu Nie, Fei Shao, Haoyu Wang, Ding Li, Xiapu Luo, and Xusheng Xiao. 2023. DISTDET: A Cost-Effective Distributed Cyber Threat Detection System. USENIX Association. <https://www.usenix.org/conference/usenixsecurity23/presentation/dong-feng>
- [9] Pengcheng Fang, Peng Gao, Changlin Liu, Erman Ayday, Kangkook Jee, Ting Wang, Yanfang Fanny Ye, Zhuotao Liu, and Xusheng Xiao. 2022. {Back-Propagating} System Dependency Impact for Attack Investigation. In *31st USENIX Security Symposium (USENIX Security 22)*. 2461–2478.
- [10] Peng Fei, Zhou Li, Zhiying Wang, Xiao Yu, Ding Li, and Kangkook Jee. 2021. SEAL: Storage-efficient Causality Analysis on Enterprise Logs with Query-friendly Compression. In *USENIX Security Symposium*. 2987–3004.
- [11] Goetz Graefe and Leonard D Shapiro. 1990. *Data compression and database performance*. University of Colorado, Boulder, Department of Computer Science.
- [12] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [13] Xueyuan Han, Thomas F. J.-M. Pasquier, Adam Bates, James Mickens, and Margo I. Seltzer. 2020. Unicorn: Runtime Provenance-Based Detector for Advanced Persistent Threats. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/unicorn-runtime-provenance-based-detector-for-advanced-persistent-threats/>
- [14] Wajih Ul Hassan, Lemay Aguse, Nuraini Aguse, Adam Bates, and Thomas Moyer. 2018. Towards scalable cluster auditing through grammatical inference over provenance graphs. In *Network and Distributed Systems Security Symposium*.
- [15] Wajih Ul Hassan, Adam Bates, and Daniel Marino. 2020. Tactical Provenance Analysis for Endpoint Detection and Response Systems. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18–21, 2020*. IEEE, 1172–1189. <https://doi.org/10.1109/SP40000.2020.00096>
- [16] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. NoDoze: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24–27, 2019*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/nodoze-combatting-threat-alert-fatigue-with-automated-provenance-triage/>
- [17] Wajih Ul Hassan, Mohammad Ali Noureddine, Pubali Datta, and Adam Bates. 2020. OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis. In *Network and distributed system security symposium*.
- [18] Md Nahid Hossain, Sadegh M. Milajerd, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R. Sekar, Scott D. Stoller, and V. N. Venkatakrishnan. 2017. SLEUTH: Real-time Attack Scenario Reconstruction from COTS Audit Data. In *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16–18, 2017*, Engin Kirda and Thomas Ristenpart (Eds.). USENIX Association, 487–504. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/hossain>
- [19] Md Nahid Hossain, Sanaz Sheikhi, and R. Sekar. 2020. Combating Dependence Explosion in Forensic Analysis Using Alternative Tag Propagation Semantics. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18–21, 2020*. IEEE, 1139–1155. <https://doi.org/10.1109/SP40000.2020.00064>
- [20] Md Nahid Hossain, Junao Wang, Ofir Weiss, R. Sekar, Daniel Genkin, Boyuan He, Scott D Stoller, Gan Fang, Frank Piessens, Evan Downing, et al. 2018. {Dependence-Preserving} data compaction for scalable forensic analysis. In *27th USENIX Security Symposium (USENIX Security 18)*. 1723–1740.
- [21] David A Huffman. 1952. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* 40, 9 (1952), 1098–1101.
- [22] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. High Accuracy Attack Provenance via Binary-based Execution Partition. In *NDSS*, Vol. 16.
- [23] Muhammad Adil Inam, Yinfang Chen, Akul Goyal, Jason Liu, Jaron Mink, Noor Michael, Sneha Gaur, Adam Bates, and Wajih Ul Hassan. 2023. Sok: History is a vast early warning system: Auditing the provenance of system intrusions. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2620–2638.
- [24] Yonghui Kwon, Fei Wang, Weihang Wang, Kyu Hyung Lee, Wen-Chuan Lee, Shiqing Ma, Xiangyu Zhang, Dongyan Xu, Somesh Jha, Gabriela F Ciocarlie, et al. 2018. MCI: Modeling-based Causality Inference in Audit Logging for Attack Investigation. In *NDSS*, Vol. 2. 4.
- [25] Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2013. Loggc: garbage collecting audit log. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 1005–1016.
- [26] Shiqing Ma, Juan Zhai, Fei Wang, Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. 2017. MPI: Multiple Perspective Attack Investigation with Semantic Aware Execution Partitioning. In *USENIX Security Symposium*. 1111–1128.
- [27] Shiqing Ma, Xiangyu Zhang, and Dongyan Xu. 2016. Protracer: Towards practical provenance tracing by alternating between logging and tainting. In *23rd Annual Network And Distributed System Security Symposium (NDSS 2016)*. Internet Soc.
- [28] Emaad Manzoor, Sadegh M. Milajerd, and Leman Akoglu. 2016. Fast Memory-Efficient Anomaly Detection in Streaming Heterogeneous Graphs. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (San Francisco, California, USA) (KDD '16)*. Association for Computing Machinery, New York, NY, USA, 1035–1044. <https://doi.org/10.1145/2939672.2939783>
- [29] Noor Michael, Jaron Mink, Jason Liu, Sneha Gaur, Wajih Ul Hassan, and Adam Bates. 2020. On the forensic validity of approximated audit logs. In *Annual Computer Security Applications Conference*. 189–202.
- [30] Sadegh M. Milajerd, Birhanu Eshete, Rigel Gjomemo, and V. N. Venkatakrishnan. 2019. POIROT: Aligning Attack Behavior with Kernel Audit Records for Cyber Threat Hunting. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11–15, 2019*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM, 1813–1830. <https://doi.org/10.1145/3319535.3363217>
- [31] Sadegh Momeni Milajerd, Rigel Gjomemo, Birhanu Eshete, R. Sekar, and V. N. Venkatakrishnan. 2019. HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows. In *2019 IEEE Symposium on Security and Privacy, SP 2019, San Francisco, CA, USA, May 19–23, 2019*. IEEE, 1137–1152. <https://doi.org/10.1109/SP.2019.00026>
- [32] Thomas F. J.-M. Pasquier, Xueyuan Han, Mark Goldstein, Thomas Moyer, David M. Eysers, Margo I. Seltzer, and Jean Bacon. 2017. Practical whole-system provenance capture. In *Proceedings of the 2017 Symposium on Cloud Computing, SoCC 2017, Santa Clara, CA, USA, September 24–27, 2017*. ACM, 405–418. <https://doi.org/10.1145/3127479.3129249>
- [33] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. FLASH: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 139–139.
- [34] Yun Shen and Gianluca Stringhini. 2019. Attack2vec: Leveraging temporal word embeddings to understand the evolution of cyberattacks. *arXiv preprint arXiv:1905.12590* (2019).
- [35] Yutao Tang, Ding Li, Zhichun Li, Mu Zhang, Kangkook Jee, Xusheng Xiao, Zhenyu Wu, Junghwan Rhee, Fengyuan Xu, and Qun Li. 2018. Nodemerge: Template based efficient data reduction for big-data causality analysis. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 1324–1337.
- [36] Colin Tankard. 2011. Advanced persistent threats and how to monitor and deter them. *Network security* 2011, 8 (2011), 16–19.
- [37] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A. Gunter, and Haifeng Chen. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/you-are-what-you-do-hunting-stealthy-malware-via-data-provenance-analysis/>
- [38] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. 2022. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Transactions on Information Forensics and Security* 17 (2022), 3972–3987.
- [39] Chunlin Xiong, Tiantian Zhu, Weihao Dong, Linqi Ruan, Runqing Yang, Yueqiang Cheng, Yan Chen, Shuai Cheng, and Xutong Chen. 2020. CONAN: A practical real-time APT detection system with high accuracy and efficiency. *IEEE Transactions on Dependable and Secure Computing* 19, 1 (2020), 551–565.
- [40] Zhiqiang Xu, Pengcheng Fang, Changlin Liu, Xusheng Xiao, Yu Wen, and Dan Meng. 2022. Depcomm: Graph summarization on system audit logs for attack investigation. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 540–557.
- [41] Zhang Xu, Zhenyu Wu, Zhichun Li, Kangkook Jee, Junghwan Rhee, Xusheng Xiao, Fengyuan Xu, Haining Wang, and Guofei Jiang. 2016. High fidelity data reduction for big data security dependency analyses. In *Proceedings of the 2016*

- ACM SIGSAC conference on computer and communications security*. 504–516.
- [42] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PROGRAPHER: An Anomaly Detection System based on Provenance Graph Embedding. USENIX Association. <https://www.usenix.org/conference/usenixsecurity23/presentation/yang-fan>
- [43] Le Yu, Shiqing Ma, Zhuo Zhang, Guan hong Tao, Xiangyu Zhang, Dongyan Xu, Vincent E Urias, Han Wei Lin, Gabriela F Ciocarlie, Vinod Yegneswaran, et al. 2021. ALchemist: Fusing Application and Audit Logs for Precise Attack Provenance without Instrumentation.. In *NDSS*.
- [44] Jun Zeng, Zheng Leong Chua, Yinfang Chen, Kaihang Ji, Zhenkai Liang, and Jian Mao. 2021. WATSON: Abstracting Behaviors from Audit Logs via Aggregation of Contextual Semantics.. In *NDSS*.
- [45] Jun Zeng, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. 2022. Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 489–506.
- [46] Tiantian Zhu, Jiayu Wang, Linqi Ruan, Chunlin Xiong, Jinkai Yu, Yaosheng Li, Yan Chen, Mingqi Lv, and Tieming Chen. 2021. General, efficient, and real-time data compaction strategy for apt forensic analysis. *IEEE Transactions on Information Forensics and Security* 16 (2021), 3312–3325.

Received 15 April 2024; revised 24 June 2024; accepted 22 July 2024